Lazy & IO

Lazy/IO · Defer expensive report until edge · Build with Waqar

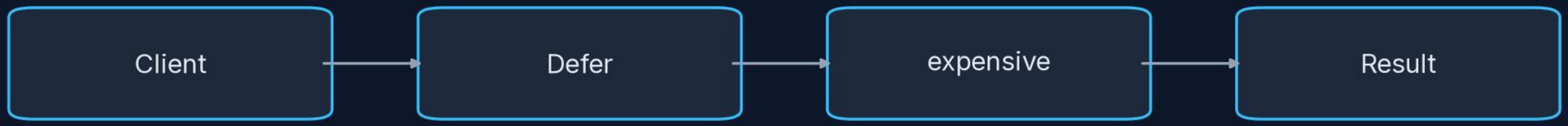
Today's Hook

- Problem: Defer expensive report until edge
- Pattern: Lazy/IO — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Defer expensive report until edge
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Lazy/IO: Defer expensive report until edge

Real-World Scenario

- Running example: Defer expensive report until edge
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.functional.lazy;

import com.buildwithwaqar.patterns.functional.lazy.LazyDemo;
import java.util.concurrent.atomic.AtomicInteger;
/** Problem: defer expensive report generation until someone asks. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Lazy problem: deferred report ===");
    var hits = new AtomicInteger();
    var lazy = LazyDemo.expensive(hits);
    System.out.println("before=" + hits.get());
    System.out.println("v=" + lazy.get() + " after=" + hits.get() + " again=" + lazy.get() + " hits=" + hits.get());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.functional.lazy;

import com.buildwithwaqar.patterns.functional.lazy.LazyDemo;
import java.util.concurrent.atomic.AtomicInteger;
/** Problem: defer expensive report generation until someone asks. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Lazy problem: deferred report ===");
    var hits = new AtomicInteger();
    var lazy = LazyDemo.expensive(hits);
    System.out.println("before=" + hits.get());
    System.out.println("v=" + lazy.get() + " after=" + hits.get() + " again=" + lazy.get() + " hits=" + hits.get());
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.lazy;

import com.buildwithwaqar.patterns.functional.lazy.LazyDemo;
import java.util.concurrent.atomic.AtomicInteger;
/** Problem: defer expensive report generation until someone asks. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Lazy problem: deferred report ===");
    var hits = new AtomicInteger();
    var lazy = LazyDemo.expensive(hits);
    System.out.println("before=" + hits.get());
    System.out.println("v=" + lazy.get() + " after=" + hits.get() + " again=" + lazy.get() + " hits=" + hits.get());
  }
}
```

Walkthrough Output

```
=== Lazy problem: deferred report ===
before=0
v=42 after=1 again=42 hits=1
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Lazy/IO.

Related Patterns

- Contrast Lazy/IO with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Lazy/IO → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.